

Test Case Prioritization based on Varying Testing Requirement Priorities and Test Case Costs

Xiaofang Zhang Changhai Nie Baowen Xu Bo Qu

School of Computer Science and Engineering, Southeast University, China

Jiangsu Institute of Software Quality

{xfzhang, changhainie, bwxu, boqu}@seu.edu.cn

Phone: (+8625)5209 0882

Abstract

Test case prioritization is an effective and practical technique in regression testing. It schedules test cases in order of precedence that increases their ability to meet some performance goals, such as code coverage, rate of fault detection. In previous work, the test case prioritization techniques and metrics usually assumed that testing requirement priorities and test case costs are uniform. In this paper, basing on varying testing requirement priorities and test case costs, we present a new, general test case prioritization technique and an associated metric. The case study illustrates that the rate of “units-of-testing-requirement-priority-satisfied-per-unit-test-case-cost” can be increased, and then the testing quality and customer satisfaction can be improved.

Keywords: Software Testing, Regression Testing, Test Case Prioritization, Priority, Cost

1. Introduction

Regression testing is an expensive testing process used to validate new features and detect regression faults, which occurs continuously during the software development lifecycle. Software engineers often save test suites so that they can reuse them later during regression testing. However, due to time and resource constraints, it may not be possible to execute all the test cases on every testing iteration [10, 14]. Therefore, testers may want to order the test cases such that those

with the higher priorities, according to some criterion, are run earlier than those with lower priorities.

Test case prioritization techniques schedule test cases in an order that increases their effectiveness at meeting some performance goals, such as code coverage, rate of fault detection. In other words, a prioritization technique is used to increase the likelihood that the prioritized test suite can better meet the goal than would a random order of test cases.

Numerous prioritization techniques have been presented in the research literature [2-7, 9-12, 14-23]. The first dimension along which these techniques can be distinguished is in terms of the type of code elements they consider. For example, statements, basic blocks, or functions. The second dimension involves the use of “feedback”. For example, additional function coverage prioritization adjusts the coverage information for remaining test cases using the feedback information. The third dimension is the use of other sources of information. For example, test cost, fault severity or fault propagation probability [5]. To measure the performance of prioritized test suite, some metrics are provided, such as total statement coverage, additional function coverage, rate of fault detection and so on.

Most of the prior techniques relied on the assumption that testing requirement priorities and test case costs are uniform. However, in practice, testing requirement priorities and test case costs can vary widely. On one side, the testing requirement priorities are changing frequently during software development, and the uniformly categorized testing requirements often fail to address stakeholder values [1, 8, 13]. On another side, test cases usually require different execution time and resources. Obviously, testing requirement priorities and test case costs should have a great impact on the test case prioritization. As a result, the existing prioritization techniques and the corresponding metrics should be

*This work was supported in part by the National Natural Science Foundation of China (60425206, 60633010, 60403016, 60503033), Natural Science Foundation of Jiangsu Province (BK2005060), High Technology Research Project of Jiangsu Province (BG2005032), and Excellent Talent Foundation on Teaching and Research of Southeast University.

adapted to incorporate the varying testing requirement priorities and test case costs.

In this paper, we propose a new, general test case prioritization technique and associated metric based on varying testing requirement priorities and test case costs. Whenever there is no ambiguity, we simply use “requirement priorities”, “test cost” instead of “testing requirement priorities”, “test case cost” respectively. The rest of this paper is organized as follows. Section 2 discusses the existing methods. Our test case prioritization technique and associated metric are proposed in section 3. Section 4 presents some valuable empirical data of case study. Finally, in section 5, it is conclusions and future work.

2. Backgrounds

2.1 Related work

Elbaum et al formally defined the *test case prioritization* problem as follows [2]:

Given: T , a test suite; PT , the set of permutations of T ; f , a function from PT to the real numbers.

Problem: Find $T' \in PT$ such that

$$(\forall T'') (T'' \in PT) (T' \neq T'') [f(T') \geq f(T'')].$$

Here, PT is the set of all possible orders of T , and f is an objective function that, applied to any such order, yields an award value for that order.

Various techniques and empirical studies have been reported on prioritizing test cases in recent years [2-7, 9-12, 14-23]. Wong et al proposed a hybrid technique that combines modification, minimization and prioritization-based selection using source code and test history [21]. Elbaum and Rothermel studied different prioritization techniques based on source code and test history, and indicated that prioritization techniques can significantly improve rate of fault detection [2-5, 15-16]. Srivastava et al described a test case prioritization technique based on basic block coverage for large systems [19]. Kim et al modeled regression testing as an ordered sequence of testing sessions and presented a history based prioritization technique utilizing the information from previous testing [9]. Walcott et al presented a prioritization approach incorporates a testing time budget [20]. Jeffrey et al proposed the test case prioritization using relevant slices [7]. Li et al discussed search algorithms for test case prioritization [11].

To formally illustrate how rapidly a prioritized test suite detects faults, Rothermel introduced a metric called *APFD* to measure the weighted average of the percentage of faults detected during the execution of the test suite [15]. Let T be a test suite which contains n

test cases, and let F be a set of m faults revealed by T . Let TF_i be the first test case in ordering T' of T which reveals fault i . The *APFD* for test suite T' could be given by the equation:

$$APFD = 1 - \frac{TF_1 + TF_2 + \dots + TF_m}{nm} + \frac{1}{2n} \quad (1)$$

APFD value ranges from 0 to 100%. An ordered test suite with higher *APFD* value has faster (better) fault detection rate than those with lower *APFD* values.

Elbaum et al proposed an improved metric *APFD_C* for measuring rate of fault detection that allows for varying test costs and fault severities [3, 12].

2.2 Discussion

Although successful in application, the existing test case prioritization techniques and metrics relied on the assumption that testing requirement priorities and test case costs are uniform. In practice, both of them can vary widely. Therefore, the existing research should be improved as follows: (1) to account for varying testing requirement priorities; (2) to consider varying test case costs; (3) to think over a suitable metric that incorporates varying testing requirement priorities and test case costs. Srikanth et al proposed a test case prioritization technique based on requirements [17-18]. However, the technique did not allow for varying test case costs. Additionally, without the consideration of feedback information, the priorities of test cases cannot be adjusted dynamically.

In testing practice, a tradeoff exists between the costs of testing and the costs of leaving unsatisfied testing requirements. Testers face and make decisions about this tradeoff frequently. It is thus appropriate that this tradeoff be considered when prioritizing test cases. In addition, a metric for evaluating prioritized test suite should reward test case orders proportionally to their rate of “units-of-testing-requirement-priority-satisfied-per-unit-test-case-cost”. In this paper, we present the test case prioritization technique based on varying testing requirement priorities and test case costs (denoted as *TCP_{RP_TC}*), and propose the associated metric *M_{RP_TC}*.

3. Prioritization technique and metric

In this section, two types of *TCP_{RP_TC}* are proposed, and then the metric *M_{RP_TC}* is presented. Next, we discuss the estimation of requirement priority and test cost in 3.3. Finally, an example is provided to indicate *TCP_{RP_TC}* and *M_{RP_TC}*.

3.1 Test case prioritization technique

The information in test history can be applied to evaluate the priorities of test cases during test case prioritization. Suppose the set of testing requirements $R=\{r_1, r_2, \dots, r_m\}$, test suite $T=\{t_1, t_2, \dots, t_n\}$, we assume that for every requirement $r \in R$, there is at least one test case t in T that satisfies r . In this study, testing requirements are determined by testing objective. Testing requirements can be different types of code elements, such as statements, basic blocks, functions; or features and attributions of system; or faults in system. Then, the test history includes:

(1) The satisfiability relation of T and R , denoted as $Sat(T, R)_{n \times m}$: if t_j satisfies r_i , then $Sat(t_j, r_i)=1$, otherwise $Sat(t_j, r_i) = 0$, here $1 \leq i \leq m$, $1 \leq j \leq n$.

(2) Test history of r_i , denoted as $r_iHistory(RH_1, RH_2, \dots, RH_s)$, where $RH_i(1 \leq i \leq s)$ is the term of $r_iHistory$, such as requirement volatility, fault proneness of the requirement, customer-assigned priority.

(3) Test history of t_j , denoted as $t_jHistory(TH_1, TH_2, \dots, TH_t)$, where $TH_i(1 \leq i \leq t)$ is the term of $t_jHistory$, such as execution time, the number of faults detected.

Basing on $Sat(T, R)_{n \times m}$, we use $Test(r_i)$ to denote the set of all test cases that can satisfy requirement r_i , where $Test(r_i)=\{t | Sat(t, r_i) = 1\}$. Similarly, $Req(t_j)$ is used to denote the set of all requirements that satisfied by test case t_j , where $Req(t_j)=\{r | Sat(t_j, r) = 1\}$. Moreover, Test history of R , T can be calculated by $r_iHistory$, $t_jHistory$ respectively, where $RHistory = \bigcup_{i=1}^m r_iHistory$, $THistory$

$$= \bigcup_{j=1}^n t_jHistory.$$

Making use of test history, we propose two types of TCP_RP_TC by adapting the existing techniques to account for the varying testing requirement priorities and test case costs: $TCP_RP_TC_Addtl$ is based on the additional requirement coverage prioritization technique while $TCP_RP_TC_Total$ is based on the total requirement coverage prioritization technique.

By adapting the additional requirement coverage prioritization, $TCP_RP_TC_Addtl$ adjusts the coverage information for remaining test cases, and iteratively selects a test case that yields the greatest coverage of requirement priorities not yet covered. Suppose $AddtlRPriority(t_j)$ is the ratio of unsatisfied requirement priorities covered by t_j to the total requirement priorities not yet covered, $TCost(t_j)$ is the test case cost of t_j , then the priority of test case t_j , denoted as $TPriority(t_j)$, is:

$$TPriority(t_j) = AddtlRPriority(t_j) / TCost(t_j) \quad (2)$$

Clearly, the larger $AddtlRPriority(t_j)$ and the smaller $TCost(t_j)$ is, then the larger $TPriority(t_j)$ is.

When the requirement priorities are not considered, $AddtlRPriority(t_j)$ is equal to the ratio of number of unsatisfied requirements covered by t_j to the total number of requirements not yet covered, denoted as $AddtlRCov(t_j)$; When all the test costs are equal, $TPriority(t_j)$ is decided by $AddtlRPriority(t_j)$; when both requirement priorities and test costs are identical, $TPriority(t_j)$ only depends on $AddtlRCov(t_j)$, in this situation, $TCP_RP_TC_Addtl$ is equivalent to the conventional technique that based on additional requirement coverage. The relationship of these four situations mentioned above is displayed in Figure 1.

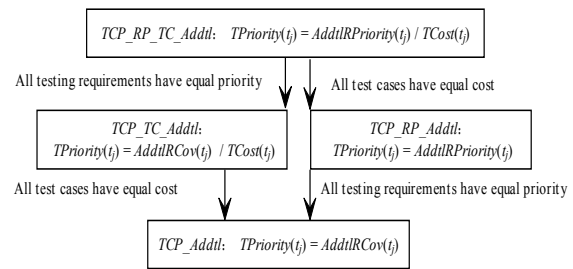


Figure 1 The relationship of four situations in $TCP_RP_TC_Addtl$

Given the priorities of test cases, $TCP_RP_TC_Addtl$ will order the test cases using *Duple-Sort* described in Algorithm 1: firstly, sort T in descending order based on the priorities of each test case; if there exists some test cases that have equal priority, then sort these test cases in ascending order based on the $|Req(t)|$ of each test case. Namely, when the priorities are equal, the test case with the smaller $|Req(t)|$, which covers the higher requirement priorities should be selected first. In this way, the requirements with the higher priorities will be satisfied as soon as possible, which is benefit to the customer satisfaction. Obviously, the worst time complexity of *Duple-Sort* is $O(n^2)$.

Algorithm 1 *Duple-Sort*

Input: $T=\{t_1, t_2, \dots, t_n\}$,

$TPriority[n]$: set of priorities for each test case,

$Req[n]$: set of requirements that each test case satisfies

Output: $SortedT$: prioritized test suite

1. $SortedT := Sort(T, TPriority[n]);$ //descending order
2. **if** (exists test cases that have equal priority in $SortedT$) **then**
3. $SortedT := Sort(SortedT, |Req[n]|);$ //ascending order
4. **endif**

The algorithm of $TCP_RP_TC_Addtl$ is shown in Algorithm 2, its time complexity is analyzed as follows: line 3-13 calculate prioritized test suite $T'LIST$ until all requirements have been covered at least once, the *while*

loop executes m times at most; the time required in line 4-7 to calculate priorities of test cases is $O(mn)$, and the time required in lines 8 to sort the test suite is $O(n^2)$ in the worst case; thus, the time complexity of line 3-13 is $O(n^3)$. The time required in line 14-16 to sort the remaining test cases is no more than $O(n^2)$, and the time required in other lines is less than $O(n^3)$. Therefore, the overall worst time complexity is $O(n^3)$.

Algorithm 2 TCP_RP_TC_Addtl

Input: $R=\{r_1, r_2, \dots, r_m\}$, $T=\{t_1, t_2, \dots, t_n\}$, $Sat(T,R)_{n \times m}$, $RHistory$, $THistory$

Output: $T'LIST$: prioritized test suite

1. $T'LIST := \Phi$;
2. calculate requirement priorities and test costs from $RHistory$, $THistory$ respectively;
3. **while** ($R \neq \Phi$) **do**
4. **for each** $t_j \in T$
5. calculate $Req(t_j)$ using $Sat(T,R)_{n \times m}$;
6. calculate $TPriority(t_j)$ using Equation 2;
7. **endfor**
8. $SortedT := Duple-Sort(T, TPriority[n], Req[n])$;
9. select the first test case t_k in $SortedT$;
10. $R = R - Req(t_k)$;
11. $T = T - \{t_k\}$;
12. $T'LIST = T'LIST + \{t_k\}$;
13. **endwhile**
14. **if** ($T \neq \Phi$) **then**
15. calculate T' by sorting T in ascending order based on $TCost(t)$;
16. **endif**
17. $T'LIST = T'LIST + T'$;

Note that, in $TCP_RP_TC_Addtl$, various strategies can be used to sort the remaining test cases after all requirements have been covered at least once. In our study, we simply sort these test cases in ascending order based on the cost of each test case, in order to execute as many test cases as possible in a limited amount of time.

Similar to $TCP_RP_TC_Addtl$, $TCP_RP_TC_Total$ calculates $TPriority(t_j)$ using test history. Suppose $TotalRPriority(t_j)$ is the ratio of requirement priorities covered by t_j to the total requirement priorities, $TCost(t_j)$ is the test case cost of t_j , then the priority of test case t_j , denoted as $TPriority(t_j)$, is:

$$TPriority(t_j) = TotalRPriority(t_j) / TCost(t_j) \quad (3)$$

In contrast, adapted from the total requirement coverage prioritization, $TCP_RP_TC_Total$ does not adjust the coverage information for remaining test cases; it merely prioritizes test cases by *Duple-Sort* for one time. As shown in Algorithm 3, the worst time complexity of $TCP_RP_TC_Total$ is $O(n^3)$.

Algorithm 3 TCP_RP_TC_Total

Input: $R=\{r_1, r_2, \dots, r_m\}$, $T=\{t_1, t_2, \dots, t_n\}$, $Sat(T,R)_{n \times m}$, $RHistory$, $THistory$

Output: $T'LIST$: prioritized test suite

1. $T'LIST := \Phi$;
2. calculate requirement priorities and test costs from $RHistory$, $THistory$ respectively;
3. **for each** $t_j \in T$
4. calculate $Req(t_j)$ using $Sat(T,R)_{n \times m}$;
5. calculate $TPriority(t_j)$ using Equation 3;
6. **endfor**
7. $SortedT := Duple-Sort(T, TPriority[n], Req[n])$;
8. $T'LIST = SortedT$;

3.2 Measuring effectiveness

To measure the performance of prioritized test suite using TCP_RP_TC , we bring forward the metric M_{RP_TC} to evaluate the rate of “units-of-testing-requirement-priority-satisfied-per-unit-test-case-cost”.

Let the horizontal axis denote “Percentage total test cost incurred”, the vertical axis denote “Percentage total requirement priority satisfied”. Under this interpretation, a test case’s contribution is weighted along the horizontal dimension in terms of test cost, and along the vertical dimension in terms of the cumulative requirement priorities it satisfied. Suppose test case t incurs 20% of total test cost and satisfies 30% of total requirement priority, then it will result in a straight line with the slope 1.5 in our M_{RP_TC} metric. In such graphs, the curve delimits a greater area for a test case order that exhibits a greater rate of “units-of-testing-requirement-priority-satisfied-per-unit-test-case-cost”.

M_{RP_TC} metric can be quantitatively described as follows: Let T be a test suite containing n test cases with test costs $tc_1, tc_2, \dots, tc_j, \dots, tc_n$, Let R be a set of requirements satisfied by T , and let $rp_1, rp_2, \dots, rp_i, \dots, rp_m$ be the priorities of these requirements. Let TR_i be the first test case in an ordering T' of T that satisfies r_i . Then, the M_{RP_TC} value of prioritized test suite T' is given by the equation:

$$M_{RP_TC} = \frac{\sum_{i=1}^m (rp_i \times (\sum_{j=TR_i}^n tc_j - \frac{1}{2} tc_{TR_i}))}{\sum_{i=1}^m rp_i \times \sum_{j=1}^n tc_j} \quad (4)$$

M_{RP_TC} values range from 0 to 100%. The higher M_{RP_TC} value indicates the better performance of T' .

Equation 4 remains applicable when either requirement priorities or test costs are identical. Furthermore, when both requirement priorities and test costs are identical ($\forall i \ rp_i = rp$ and $\forall i \ tc_i = tc$), the formula can be simplified to:

$$M_{RP_TC} = 1 - \frac{TR_1 + TR_2 + \dots + TR_m}{mn} + \frac{1}{2n} \quad (5)$$

From Equation 4 and 5, we can find out that: $APFD_C$ [3, 12] and M_{RP_TC} evaluate the performance of prioritized test suite from different profiles respectively. $APFD_C$ represents the rate of “units-of-fault-severity-detected-per-unit-test-case-cost”, while our M_{RP_TC} indicates the rate of “units-of-testing-requirement-priority-satisfied-per-unit-test-case-cost”. Suppose the testing requirement priority only depends on its associated fault severities, $APFD_C$ can be regarded as one of the particular cases of M_{RP_TC} .

Different test case prioritization techniques will result in different M_{RP_TC} values. An increased M_{RP_TC} will improve testing quality and customer satisfaction. Firstly, it provides earlier feedback on the system under test, supporting faster strategic decisions about release schedules. Moreover, if the testing period is cut short, it can increase the possibility of executing those test cases that cover the higher testing requirement priorities during the available testing time.

3.3 Estimating testing requirement priority and test case cost

In TCP_RP_TC and M_{RP_TC} , the estimation of requirement priorities and test costs is required. As for TCP_RP_TC , it is a prioritization technique that needs to predict requirement priorities and test costs before test suite execution. However, the prediction is usually difficult in practice. One solution is to rely on test history gathered during a preceding testing session. Under this approach, it assumes that the requirement priorities and test costs do not change significantly from one release to another. In terms of M_{RP_TC} , which occurs after test suite execution, the requirement priorities and test costs can be simply obtained from the current test history.

In this context, requirement priority and test cost can be estimated in various ways.

Requirement priority reflects different importance of each requirement in testing. It can be calculated by influence factors and factor weight. For each testing object, users can customize the influence factors and factor weight. Suppose there are s influence factors, $RFactorWeight_k$ is the weight for factor k ($1 \leq k \leq s$), let

$RFactorWeight_k \in [0, 1]$, and $\sum_{k=1}^s RFactorWeight_k = 1$,

$RFactorValue_{ik}$ is the value of factor k for requirement r_i , then the priority of r_i , denoted as $RPriority(r_i)$, can be calculated as follows:

$$RPriority(r_i) = \sum_{k=1}^s (RFactorWeight_k * RFactorValue_{ik}) \quad (6)$$

Higher requirement priority corresponds to a higher number, and implies a higher importance for this requirement to be tested. In this paper, we primarily consider the following three influence factors: requirement volatility (RH_1), fault proneness of the requirement (RH_2), customer-assigned priority (RH_3), where RH_1 and RH_2 can be obtained from test history.

In terms of test case cost, it is related to the resources, such as execution time of test case, hardware costs or even engineers' salaries. In this paper, we primarily account for the execution time of test case.

3.4 An example

An example is utilized to display the process of TCP_RP_TC and M_{RP_TC} . Suppose $R = \{r_1, r_2, \dots, r_6\}$, $T = \{t_1, t_2, t_3, t_4\}$, $Sat(T, R)_{n \times m}$ is shown in Table 1, $RFactorWeight$ and $RFactorValue$ are shown in Table 2. Then, the priorities of R are calculated and recorded in $RPriority[6] = \{6, 9, 7, 6, 3, 3\}$. Given the test case costs of T are $TCost[4] = \{2, 3, 2, 3\}$, we can calculate the prioritized test suite and M_{RP_TC} by the following three different techniques.

Table 1 The satisfiability relation of T and R

	r_1	r_2	r_3	r_4	r_5	r_6
t_1	X	X				
t_2	X		X	X		
t_3	X			X	X	
t_4	X		X			X

Table 2 $RFactorWeight$ and $RFactorValue$

$RFactorWeight$	r	r_1	r_2	r_3	r_4	r_5	r_6
0.25	RH_1	3	10	9	2	7	2
0.25	RH_2	5	8	9	8	1	2
0.5	RH_3	8	9	5	7	2	4
$RPriority$		6	9	7	6	3	3

(1) $TCP_RP_TC_Addtl$ based on Algorithm 2: as shown in Table 3, while $R = \{r_1, r_2, \dots, r_6\}$, $T = \{t_1, t_2, t_3, t_4\}$, $TPriority(t_1) = TPriority(t_3)$, $|Req(t_1)| = 2 < |Req(t_3)| = 3$, so t_1 is added to $T'LIST$ at first; secondly, while $R = \{r_3, r_4, r_5, r_6\}$, $T = \{t_2, t_3, t_4\}$, t_3 is added to $T'LIST$; thirdly, while $R = \{r_3, r_6\}$, $T = \{t_2, t_4\}$, t_4 is added to $T'LIST$. At this moment, all of the requirements have been satisfied by $T'LIST = \{t_1, t_3, t_4\}$. Finally, the remaining test case t_2 is added, $T'LIST = \{t_1, t_3, t_4, t_2\}$ in the end. Using Equation 4, $M_{RP_TC} = 71.5\%$, as shown in Figure 2.a.

(2) $TCP_RP_TC_Total$ based on Algorithm 3: while $R = \{r_1, r_2, \dots, r_6\}$, $T = \{t_1, t_2, t_3, t_4\}$, $TPriority(t_1) = TPriority(t_3) > TPriority(t_2) > TPriority(t_4)$, $|Req(t_1)|$

$=2 < |Req(t_3)| = 3$, so $T'LIST = \{t_1, t_3, t_2, t_4\}$. In this case, $M_{RP_TC} = 68.8\%$, as shown in Figure 2.b.

(3) *Initial-Ordering*: test cases are selected and executed in the original sequence of T . Thus, $T'LIST = \{t_1, t_2, t_3, t_4\}$, $M_{RP_TC} = 69.4\%$, as shown in Figure 2.c.

Figure 2.d compares three prioritization techniques mentioned above. In this example, $M_{RP_TC}(t_1, t_3, t_4, t_2) > M_{RP_TC}(t_1, t_2, t_3, t_4) > M_{RP_TC}(t_1, t_3, t_2, t_4)$, that is to say, the prioritized performance of $TCP_RP_TC_Addtl$ is best in M_{RP_TC} metric.

Table 3 Calculation of $T'LIST$ by $TCP_RP_TC_Addtl$

	$R = \{r_1, r_2, \dots, r_6\}$ $T = \{t_1, t_2, t_3, t_4\}$	$R = \{r_3, r_4, r_5, r_6\}$ $T = \{t_2, t_3, t_4\}$	$R = \{r_3, r_6\}$ $T = \{t_2, t_4\}$
$TPriority(t_1)$	0.221	--	--
$TPriority(t_2)$	0.186	0.228	0.233
$TPriority(t_3)$	0.221	0.237	--
$TPriority(t_4)$	0.157	0.175	0.333
$T'LIST$	$\{t_1\}$	$\{t_1, t_3\}$	$\{t_1, t_3, t_4\}$

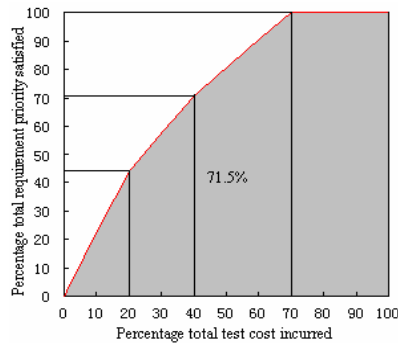


Figure 2.a $TCP_RP_TC_Addtl$

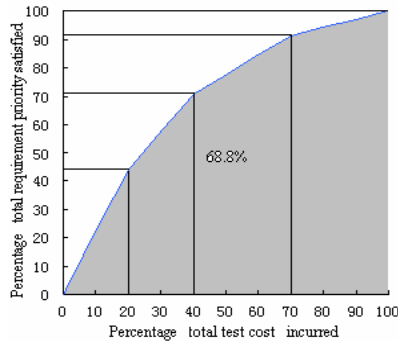


Figure 2.b $TCP_RP_TC_Total$

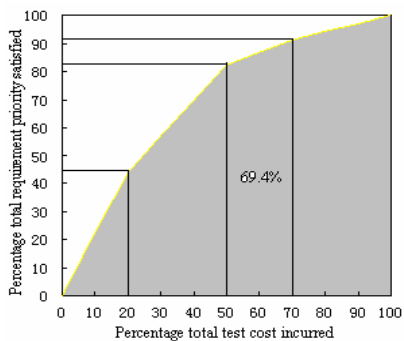


Figure 2.c *Initial-Ordering*

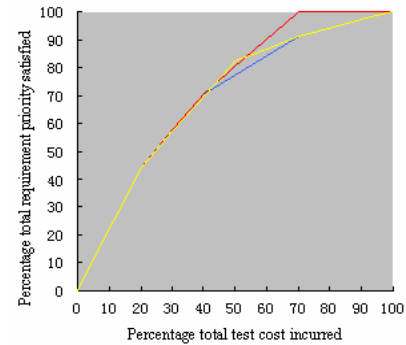


Figure 2.d Three techniques

4. Case study

A series of simulation experiments are performed to validate the effectiveness of our TCP_RP_TC and M_{RP_TC} .

4.1 Experiment design

The case study has the following three steps: (1) Generate a set of testing requirements and the corresponding test suite, construct $Sat(T, R)_{n \times m}$ randomly. (2) Estimate testing requirement priorities and test case costs. (3) Utilize three prioritization techniques respectively to calculate the prioritized test suite and corresponding M_{RP_TC} . The parameters of the empirical study include: the number of testing requirements (m), the number of test cases (n), the requirement priorities (RP), the test costs (TC) and the test case prioritization techniques (TCP). In this paper, we focus on the impact of RP , TC and TCP distributions on prioritization.

For RP , we use two requirement priority distributions:

- *Equal*: all requirement priorities are equal. It is the case in which requirement priorities are not considered.
- *Random*: requirement priorities are uniformly distributed over the range $[1 \dots 10]$.

For TC , we use four test cost distributions:

- *Equal*: all test costs are equal. It is the case in which test costs are not considered.
- *Random*: test costs are uniformly distributed over the range $[1 \dots 10]$.
- *Mozilla*: Test costs are distributed as in the Mozilla application (see Table 4) [3].
- *QTB*: Test costs are distributed as in the QTB application (see Table 5) [3].

Given these two requirement priority distributions and four test cost distributions, there are eight combined distributions of RP and TC , as shown in Table 6. To apply each distribution, we construct the

generator of *equal*, *random*, *Mozilla*, *QTB* respectively at first, then generate a set of numbers having the required distribution, and randomly map these numbers to requirements or test cases.

Table 4 Mozilla test case cost distribution

name	level	description	percentage
HTML	1	least expensive	87%
Printing	2	more expensive	1%
Smoke tests	3	expensive	2%
Buster	4	most expensive	10%

Table 5 QTB test case cost distribution

level	description	percentage
1	not expensive	88%
10	expensive	12%

Table 6 Combined distributions of RP - TC

	equal	random	Mozilla	QTB
equal	A ₁ /B ₁	A ₂	A ₃	A ₄
random	B ₂	C ₁	C ₂	C ₃

For TCP , we use three test case prioritization techniques:

• $TCP_RP_TC_Addtl$: test cases are prioritized according to Algorithm 2;

• $TCP_RP_TC_Total$: test cases are prioritized according to Algorithm 3;

• *Initial-Ordering*: test cases are prioritized in the original sequence of T .

For 10 different sets of parameters (m , n), 20 trials are simulated. In each trial, 24 combinations of (RP , TC , TCP) are considered on the randomly generated $Sat(T, R)_{n \times m}$. The average M_{RP_TC} of 20 trials are recorded in Appendix A. Then, the average M_{RP_TC} of 10 different sets of parameters (m , n) are calculated and analyzed in the following section.

4.2 Results and discussion

We present the results of our case study in three steps: (1) varying test cost distributions across different prioritization techniques while maintaining an equal requirement priority distribution (A₁~A₄ in Table 6); (2) varying requirement priority distributions across different prioritization techniques while maintaining an equal test cost distribution (B₁~B₂ in Table 6); (3) varying distributions for both requirement priority and test cost (C₁~C₃ in Table 6).

(1) Varying test cost distributions

While RP =equal, TC ={equal, random, Mozilla, QTB}, Table 7 displays the average M_{RP_TC} measured under this situation, where $All(TC)$ denotes the average M_{RP_TC} across different test cost distributions, $All(TCP)$ denotes the average M_{RP_TC} across different test case prioritization techniques. Figure 3 provides a visualized

view of the M_{RP_TC} values. The figure displays three sets of bars, each set of bars contains five individual bars: the leftmost one for $All(TC)$, and the other four for each test cost distribution studied. The height of each bar expresses the average M_{RP_TC} under the technique and distribution associated with that bar.

Table 7 Average M_{RP_TC} for varying TC , RP =equal

RP =equal	Addtl	Total	Ini	$All(TCP)$
A ₁ : equal	0.9481	0.9005	0.9164	0.9217
A ₂ : random	0.9779	0.9249	0.9174	0.9401
A ₃ : Mozilla	0.9667	0.9283	0.9468	0.9473
A ₄ : QTB	0.9771	0.9464	0.9636	0.9624
$All(TC)$	0.9674	0.9250	0.9361	

Note: Addtl: $TCP_RP_TC_Addtl$, Total: $TCP_RP_TC_Total$, Ini: Initial-ordering

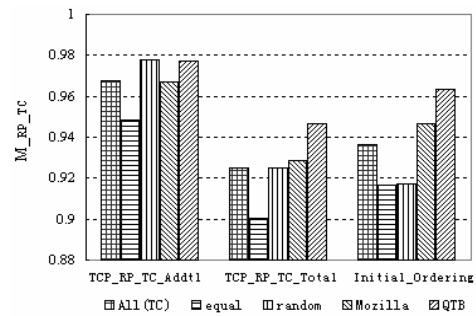


Figure 3 Average M_{RP_TC} for varying TC , RP =equal

From Table 7 and Figure 3, we discuss the extent of the impact varied with techniques (as shown by $All(TC)$). In $TCP_RP_TC_Addtl$, the average M_{RP_TC} across different distributions is 0.9674, which is statistically the largest, with 4.58% more than that of $TCP_RP_TC_Total$ and 3.34% more than that of *Initial-Ordering*. As for $TCP_RP_TC_Total$ and *Initial-Ordering*, the average M_{RP_TC} of the former is usually worse than that of the latter, except for TC =random. Moreover, test cost distributions do have an impact on the performance of prioritized test suites across all techniques (as shown by $All(TCP)$). The differences between A₂~A₄ and A₁ are statistically significant, where the average M_{RP_TC} of A₄ is 4.42% more than that of A₁.

(2) Varying requirement priority distributions

While TC =equal, RP ={equal, random}, the results are shown in Table 8. We can find out that: in $TCP_RP_TC_Addtl$, the average M_{RP_TC} across different distributions is also the largest, with 5.60% more than that of $TCP_RP_TC_Total$ and 3.62% more than that of *Initial-Ordering*. In this case, the average M_{RP_TC} of *Initial-Ordering* is better than that of $TCP_RP_TC_Total$. As for the impact of varying requirement priority distribution on the performance of prioritized test suites, it's not as large as we expected.

Compared with B_1 , the average M_{RP_TC} of $TCP_RP_TC_Addtl$ in B_2 is slightly higher, while that of *Initial-Ordering* is roughly equal, and that of $TCP_RP_TC_Total$ is slightly lower.

Table 8 Average M_{RP_TC} for varying RP , $TC = \text{equal}$

$TC = \text{equal}$	<i>Addtl</i>	<i>Total</i>	<i>Ini</i>	<i>All(TCP)</i>
$B_1: \text{equal}$	0.9481	0.9005	0.9164	0.9217
$B_2: \text{random}$	0.9510	0.8981	0.9163	0.9218
<i>All(TC)</i>	0.9496	0.8993	0.9164	

Note: *Addtl*: $TCP_RP_TC_Addtl$, *Total*: $TCP_RP_TC_Total$, *Ini*: *Initial-ordering*

(3) Varying requirement priority and test cost distributions

While $RP = \text{random}$, $TC = \{\text{random, Mozilla, QTB}\}$, the observations in Table 9 indicate that: in $TCP_RP_TC_Addtl$, the average M_{RP_TC} across different distributions is statistically the largest, with 4.48% more than that of $TCP_RP_TC_Total$ and 3.47% more than that of *Initial-Ordering*. In this case, the average M_{RP_TC} of $TCP_RP_TC_Total$ is usually worse than that of *Initial-Ordering*, except in C_1 . In addition, varying distributions for both requirement priority and test cost do have an impact on the performance of prioritized test suites. The differences between $C_1 \sim C_3$ and A_1/B_1 are statistically significant, where the average M_{RP_TC} of C_3 is 0.9628, which is 4.46% more than that of A_1/B_1 .

Table 9 Average M_{RP_TC} for varying RP_TC

RP_TC	<i>Addtl</i>	<i>Total</i>	<i>Ini</i>	<i>All(TCP)</i>
$A_1/B_1: \text{equal_equal}$	0.9481	0.9005	0.9164	0.9217
$C_1: \text{random_random}$	0.9787	0.9253	0.9170	0.9403
$C_2: \text{random_Mozilla}$	0.9684	0.9279	0.9467	0.9477
$C_3: \text{random_QTB}$	0.9783	0.9466	0.9635	0.9628
<i>All(TC)</i>	0.9751	0.9333	0.9424	

Note: *Addtl*: $TCP_RP_TC_Addtl$, *Total*: $TCP_RP_TC_Total$, *Ini*: *Initial-ordering*, *All(TC)* calculate average of $C_1 \sim C_3$ here.

In conclusion, the empirical studies above advise that: (1) the performance of $TCP_RP_TC_Addtl$ is always the best in the three test case prioritization techniques. It may be because that $TCP_RP_TC_Addtl$ adjusts the coverage information for remaining test cases, and iteratively selects a test case that yields the greatest coverage of requirement priorities not yet covered, which is helpful to improve the M_{RP_TC} value. (2) *Initial-Ordering* usually performs better than $TCP_RP_TC_Total$, which is out of our expectation. One possible reason is that the test suite by *Initial-Ordering* is more similar to the prioritized test suite by $TCP_RP_TC_Addtl$. As a result, the M_{RP_TC} value of *Initial-Ordering* is higher than that of $TCP_RP_TC_Total$. (3) compared with the case in which all requirement priorities and test costs are equal, the varying distributions for both requirement priority and test cost

actually have a significant impact on the M_{RP_TC} value. The improved M_{RP_TC} value, in other words, the increasing rate of “units-of-testing-requirement-priority-satisfied-per-unit-test-case-cost”, contributes to the enhancement of testing quality and customer satisfaction.

5. Conclusions and future work

Test case prioritization is a widely used strategy to improve regression testing. However, to date, there has been little work regarding how to incorporate varying testing requirement priorities and test case costs into this strategy. The assumption that requirement priorities and test costs are uniform is not often met in practice. To address the issue, this paper presents the following improvements:

(1) Our test case prioritization techniques, $TCP_RP_TC_Addtl$ and $TCP_RP_TC_Total$, adapted from previous work, incorporate the varying testing requirement priorities and test case costs. Other existing prioritization techniques can be adjusted in the similar way.

(2) The metric M_{RP_TC} , which evaluates the rate of “units-of-testing-requirement-priority-satisfied-per-unit-test-case-cost”, can provide the guidelines for testers to select the suitable prioritization technique.

(3) The simulation experiments empirically compare the performance in 24 combinations of requirement priorities, test costs and test case prioritization techniques. In this way, the effectiveness of our TCP_RP_TC and M_{RP_TC} are preliminary validated.

In the future, we plan to do further empirical studies to evaluate our techniques and metric, providing data to propose more rigorous guidelines. Furthermore, we will construct case studies using real systems, with requirement priority and test cost data drawn directly from those systems. Additionally, in some systems, such as the safety-critical systems, the requirement priority and test cost estimation approaches described earlier may not satisfactorily capture the characteristics. In these cases, we will study the approach to adjust the distributions of requirement priority and test cost.

References

- [1] B. Boehm. Value-Based Software Engineering. ACM SIGSOFT Software Engineering Notes, 18(2):3-7, 2003.
- [2] S. Elbaum, A. G. Malishevsky and G. Rothermel. Prioritizing Test Cases for Regression Testing. In Proceedings of the International Symposium on Software Testing and Analysis, pages 102-112, Aug 2000.

- [3] S. Elbaum, A. G. Malishevsky and G. Rothermel. Incorporating varying test costs and fault severities into test case prioritization. In Proceedings of the 23rd International Conference on Software Engineering, pages 329-338, May 2001.
- [4] S. Elbaum, A. G. Malishevsky and G. Rothermel. Test case prioritization: A family of empirical studies. IEEE Transactions on Software Engineering, 28(2): 159-182, 2002.
- [5] S. Elbaum, G. Rothermel, S. Kanduri and A. G. Malishevsky. Selecting a Cost-Effective Test Case Prioritization Technique. Software Quality Journal, 12(3): 185-210, 2004.
- [6] M. J. Harrold. Testing: A Roadmap, In Proceedings of the International Conference on Software Engineering, Limerick, Ireland, pages 61-72, 2000.
- [7] D. Jeffrey and N. Gupta. Test Case Prioritization Using Relevant Slices. In Proceedings of the 30th Annual International Computer Software and Applications Conference, pages 411-420, 2006.
- [8] J. Karlsson and K. Ryan. A Cost-Value Approach for Prioritizing Requirements, IEEE Software, 14(5): 67-74, 1997.
- [9] J. M. Kim and A. Porter. A History-Based Test Prioritization Technique for Regression Testing in Resource Constrained Environments. In Proceedings of the 24th International Conference on Software Engineering, pages 119-129, May 2002.
- [10] H. Leung and L. White. Insights into regression testing. In Proceedings of the International Conference on Software Maintenance, Miami, Florida, U.S.A., pages 60-69, Oct 1989.
- [11] Z. Li, M. Harman and R. M. Hierons. Search Algorithms for Regression Test Case Prioritization. IEEE Transactions on Software Engineering, 33(4):225-237, April 2007.
- [12] A. G. Malishevsky, J. Ruthruff, G. Rothermel, and S. Elbaum, Cost-cognizant Test Case Prioritization, Technical Report TR-UNL-CSE-2006-0004, Department of Computer Science and Engineering, University of Nebraska - Lincoln, March, 2006.
- [13] G. Mogyorodi. Requirements-Based Testing: An Overview. Starbase Corporation, 2001.
- [14] K. Onoma, W-T. Tsai, M. Poonawala and H. Saganuma. Regression testing in an industrial environment. Communications of the ACM, 41(5): 81-86, May 1998.
- [15] G. Rothermel, R. H. Untch, C. Chu and M. J. Harrold. Test Case Prioritization: An Empirical Study. In Proceedings of the International Conference on Software Maintenance, Sep 1999.
- [16] G. Rothermel, R. H. Untch, C. Chu, and M. J. Harrold. Prioritizing test cases for regression testing. IEEE Transactions of Software Engineering, 27(10): 929-948, 2001.
- [17] H. Srikanth. Requirements-Based Test Case Prioritization. Student Research Forum in 12th ACM SIGSOFT International Symposium on the Foundations of Software Engineering, 2004.
- [18] H. Srikanth and L. Williams. On the economics of requirements-based test case prioritization. In Proceedings of the 7th international workshop on Economics-driven software engineering research, pages 1-3, May 2005.
- [19] A. Srivastava and J. Thiagarajan. Effectively prioritizing tests in development environment. In Proceedings of the International Symposium on Software Testing and Analysis, pages 97-106, July 2002.
- [20] K. R. Walcott, M. L. Soffa, G. M. Kapfhammer and R. S. Roos. Time-Aware Test Suite Prioritization. In Proceedings of the International Symposium on Software testing and Analysis, pages 1-12, July 2006.
- [21] W. E. Wong, J. R. Horgan, S. London and H. Agrawal. A Study of Effective Regression Testing in Practice. In Proceedings of the 8th IEEE International Symposium on Software Reliability Engineering, pages 264-274, Nov 1997.
- [22] Xiaofang Zhang, Baowen Xu, Changhai Nie and Liang Shi. An Approach for Optimizing Test Suite Based on Testing Requirement Reduction. Journal of Software (in Chinese), 18(4): 821-831, 2007.
- [23] Xiaofang Zhang, Baowen Xu, Changhai Nie and Liang Shi. Test Suite Optimization Based on Testing Requirements Reduction. International Journal of Electronics & Computer Science, 7(1): 9-15, 2005.

Appendix A: Average M_{RP_TC} of 20 trials for 10 different sets of parameters (m, n)

m	n	e-e-add	e-e-total	e-e-ini	e-r-add	e-r-total	e-r-ini	e-m-add	e-m-total	e-m-ini	e-q-add	e-q-total	e-q-ini
20	8	0.8963	0.8250	0.8562	0.9493	0.8216	0.8569	0.9388	0.8515	0.9175	0.9509	0.8685	0.9323
10	8	0.9006	0.815	0.8487	0.9492	0.8298	0.8486	0.9428	0.8662	0.9135	0.9532	0.8830	0.9288
20	12	0.9308	0.8710	0.8912	0.9730	0.8893	0.8969	0.9531	0.9075	0.9275	0.9719	0.9347	0.9565
100	20	0.9469	0.9232	0.9312	0.9829	0.9402	0.9332	0.9632	0.9471	0.9526	0.9772	0.9664	0.9707
50	25	0.9630	0.9336	0.9412	0.9880	0.9562	0.9452	0.9748	0.9537	0.9602	0.9820	0.9671	0.9717
8	10	0.9325	0.8675	0.8768	0.9680	0.9019	0.8905	0.9574	0.9128	0.9230	0.9756	0.9421	0.9560
12	20	0.9652	0.9116	0.9258	0.9859	0.9582	0.9124	0.9752	0.9422	0.9488	0.9847	0.9643	0.9684
8	20	0.9684	0.9112	0.9343	0.9887	0.9660	0.9305	0.9782	0.9396	0.9547	0.9865	0.9627	0.9720
25	50	0.9844	0.9651	0.9725	0.9959	0.9894	0.9732	0.9886	0.9754	0.9803	0.9923	0.9831	0.9867
20	100	0.9927	0.9818	0.9863	0.9982	0.9960	0.9864	0.9946	0.9868	0.9898	0.9965	0.9914	0.9934
Average		0.9481	0.9005	0.9164	0.9779	0.9249	0.9174	0.9667	0.9283	0.9468	0.9771	0.9464	0.9636

m	n	r-e-add	r-e-total	r-e-ini	r-r-add	r-r-total	r-r-ini	r-m-add	r-m-total	r-m-ini	r-q-add	r-q-total	r-q-ini
20	8	0.9015	0.8156	0.8572	0.9506	0.8289	0.8567	0.9420	0.8502	0.9181	0.9531	0.8580	0.9328
10	8	0.9100	0.7974	0.8397	0.9504	0.8154	0.8394	0.9473	0.8610	0.9084	0.9573	0.8870	0.9246
20	12	0.9336	0.8679	0.8923	0.9764	0.8914	0.8983	0.9547	0.9043	0.9282	0.9728	0.9339	0.9569
100	20	0.9489	0.9226	0.9318	0.9829	0.9405	0.9339	0.9644	0.9461	0.9530	0.9780	0.9656	0.9710
50	25	0.9639	0.9363	0.9398	0.9881	0.9577	0.9439	0.9755	0.9551	0.9593	0.9826	0.9689	0.9710
8	10	0.9371	0.8679	0.8828	0.9683	0.9056	0.8971	0.9605	0.9156	0.9267	0.9774	0.9494	0.9581
12	20	0.9658	0.9109	0.9242	0.9866	0.9616	0.9099	0.9760	0.9419	0.9477	0.9852	0.9642	0.9677
8	20	0.9704	0.9153	0.9351	0.9891	0.9661	0.9313	0.9796	0.9423	0.9552	0.9874	0.9644	0.9723
25	50	0.9850	0.9645	0.9729	0.9961	0.9898	0.9733	0.9890	0.9752	0.9807	0.9926	0.9832	0.9870
20	100	0.9930	0.9816	0.9863	0.9983	0.9961	0.9865	0.9948	0.9867	0.9899	0.9966	0.9915	0.9934
Average		0.9510	0.8981	0.9163	0.9787	0.9253	0.9170	0.9684	0.9279	0.9467	0.9783	0.9466	0.9635

Note: e-equal, r-random, m-Mozilla, q-QTB, add-TCP_RP_TC_Addtl, total-TCP_RP_TC_Total, ini-Initial-Ordering.